Auto Mininet: LLM Based generation of python script for mininet topologies

Group Members:

Saim Haider (22P-9244)

Muhammad Abdullah (22P-9371)

Abdullah (22P-9358)

Project Supervisor

Dr. Bilal Khan

Introduction

- Mininet is widely used for network simulation, that can use python scripts to generate topologies.
- Students face difficulty converting topology concepts → valid Mininet code.
- Our project fine-tunes an instruction-tuned code LLM on Mininet datasets.
- The result: a domain-specific model that reliably generates Mininet topologies.

Problem Statement

- Current LLMs can generate generic Python code, but they are not domain-specialized for Mininet. This leads to errors, inconsistencies, and inefficiency in network labs(For learning and experimenting purposes). There is no fine-tuned AI model available for Mininet code generation.

Primary Objective

- To develop an **AI-Powered Educational Tool** that generate python scripts for custom network topologies and help students **learn network topologies easily.**

Specific Objective

- Fine-tune an existing LLM to generate accurate python scripts for **mininet**.
- Collect a dataset of diverse network topologies for model training.
- Build an interactive **web interface** for topology generation and script display.
- Evaluate the model's performance using accuracy, latency, and usability metrics.

Literature Review

Sr	Author	Methodology	Key Findings	Gaps
1	Dong-Wook Kim, Gun-YoonShin, etal. (2023)	Developed a framework to generate network topologies from an existing traffic dataset using graph theory (Dijkstra's algorithm) to find paths for traffic replay in Mininet.	Automated topology generation from a specific data source for a particular application (cyber exercises).	The generation is data-driven, based on analyzing an existing traffic dataset, not on a user's conceptual or natural language request.
2	Oleksandr I. Romanov, Ivan O. Savchenko, Anton I. Marinov, Serhii S. Skolets. (2021)	Built a simulation model in Mininet to determine performance indicators (RTT, band width, delay).	Provided deeper insights into performance testing methodologies for mininet topologies.	Explicitly stated that setting up the system requires writing programs to ensure the interaction among standard elements from the Mininet library, reinforcing the core problem.
3	Andr´es Laurito, Mat´ıas Bonaventura, Mikel Eukeni, Pozo Astigarraga, Rodrigo Castro. (2017)	Introduced TopoGen, a flexible architecture to translate network topologies from diverse sources (like an SDN controller) into various simulation model formats using an intermediate format	Created a powerful translation pipeline for interoperability between different networking tools and formats.	Remained a format to-format translator; it does not generate a network from abstract human intent.

Literature Review

Sr	Author	Methodology	Key Findings	Gaps
4	Sunit Kumar Nandi. (2016)	Proposed two topology generators for Mininet: a random topology generator (Erd˝os-R´enyi model) and a generator using a SNAP format dataset.	Automate the creation of large scale random or dataset derived topologies that are impractical to create manually.	The input was either statistical parameters or a specific dataset format (SNAP), not a high-level descriptive request from a user.
5	Marcel Großmann, Stephan J.A. Schuberth. (2013)	Developed a parser automatically generate Mininet Python scripts from graphical GraphML files provided by the Internet Topology Zoo.	Solved the problem of manually recreating complex, real-world topologies in Mininet. Bridged the gap between a graphical database and a functional script.	The automation was strictly format-to format; it required a pre-existing, structured GraphML file. •The system could not interpret a conceptual or descriptive request from a user.

References

Kim, D.-W., Shin, G.-Y., et al. (2023). *Framework for Network Topology Generation and Traffic Prediction Analytics for Cyber Exercises.* Proceedings of the International Conference on Cybersecurity and Networks.

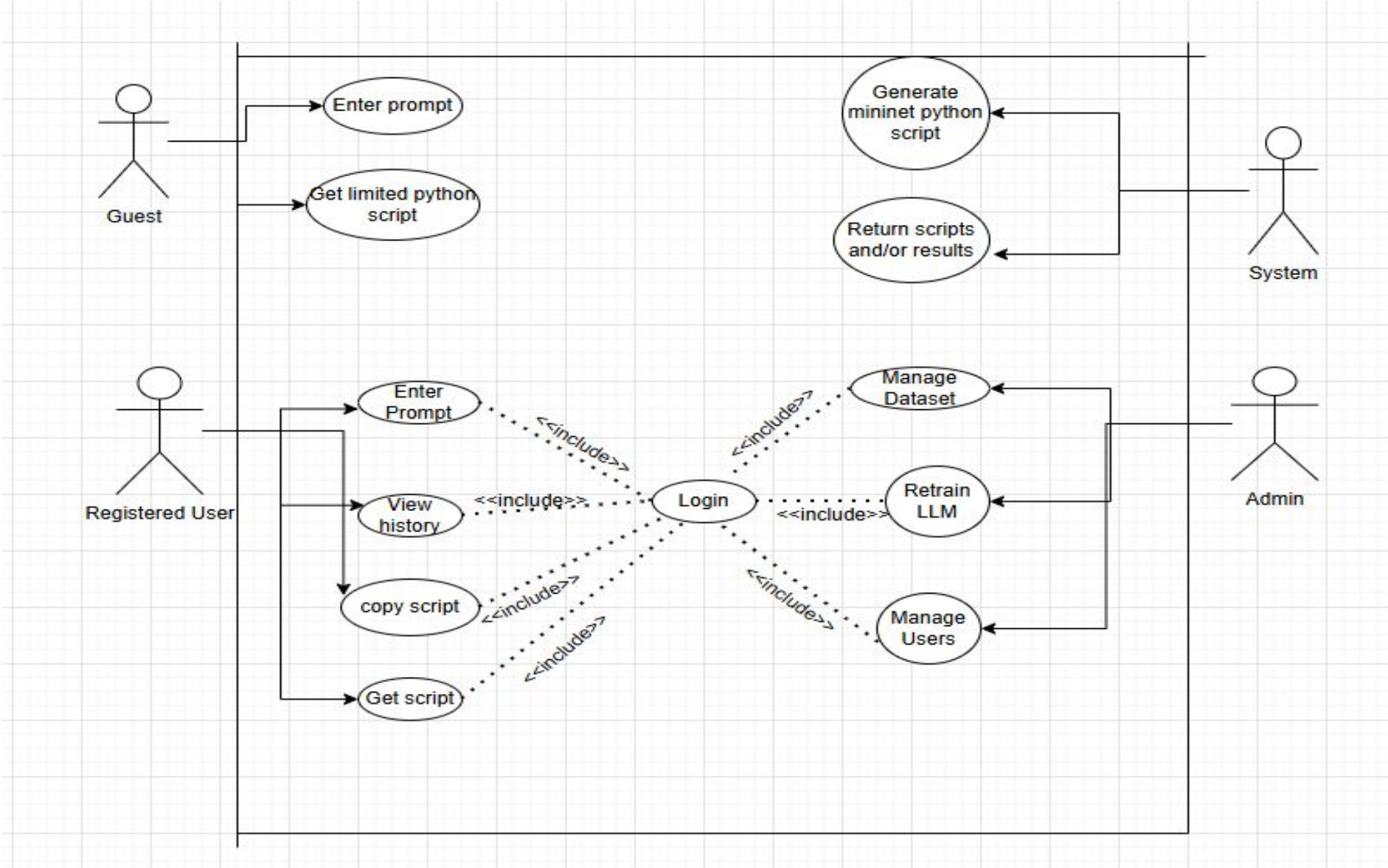
Romanov, O. I., Savchenko, I. O., Marinov, A. I., & Skolets, S. S. (2021). *Research of SDN Network Performance Parameters Using Mininet Network Emulator.* Journal of Theoretical and Applied Information Technology, 99(8), 1907–1917

Laurito, A., Bonaventura, M., Astigarraga, M. E. P., & Castro, R. (2017). *TOPOGEN: A Network Topology Generation Architecture.* Proceedings of the IEEE Symposium on Network Management.

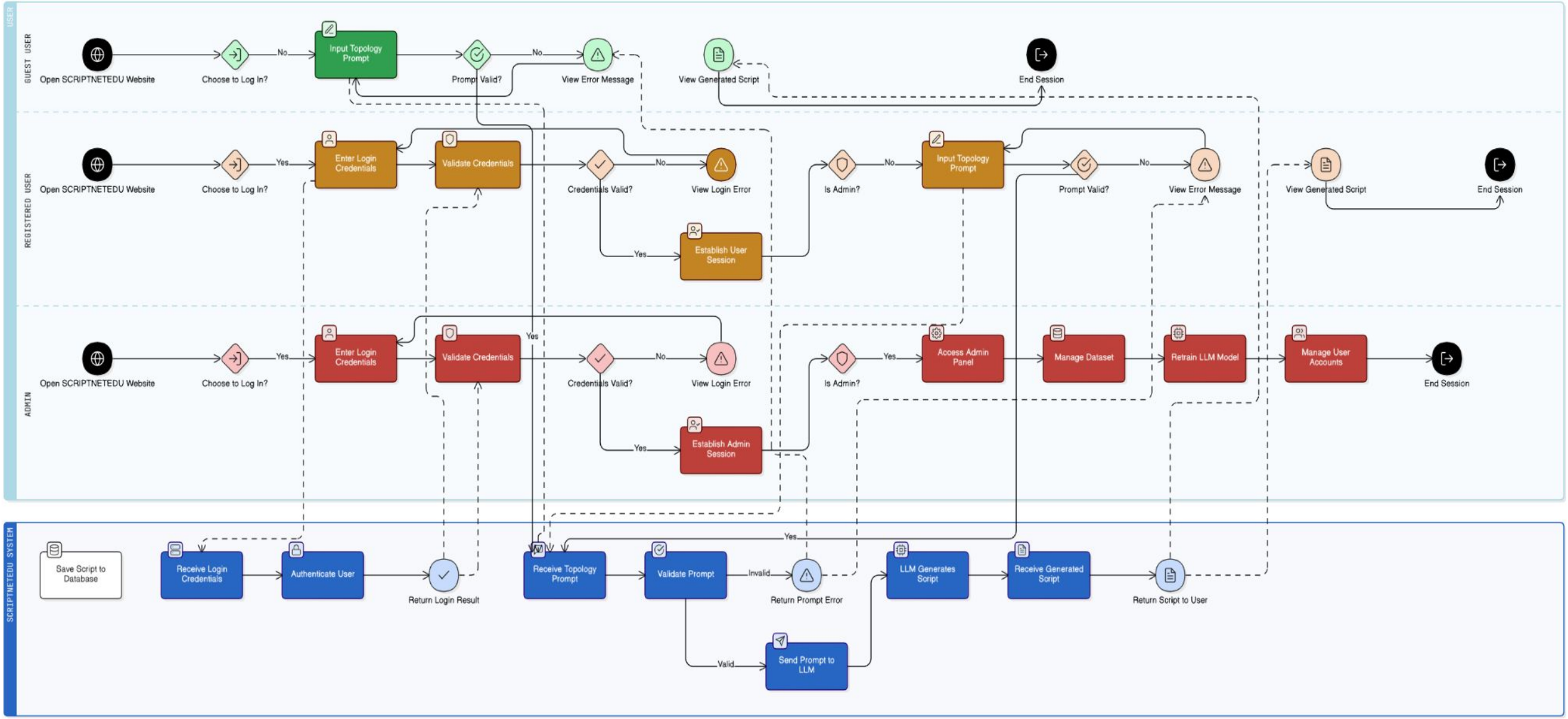
Nandi, S. K. (2016). *Topology Generators for Software Defined Network Testing.*

Großmann, M., & Schuberth, S. J. A. (2013). *Auto-Mininet: Assessing the Internet Topology Zoo in a Software-Defined Network Emulator.* Proceedings of the IEEE Conference on Network Simulation and Emulation.

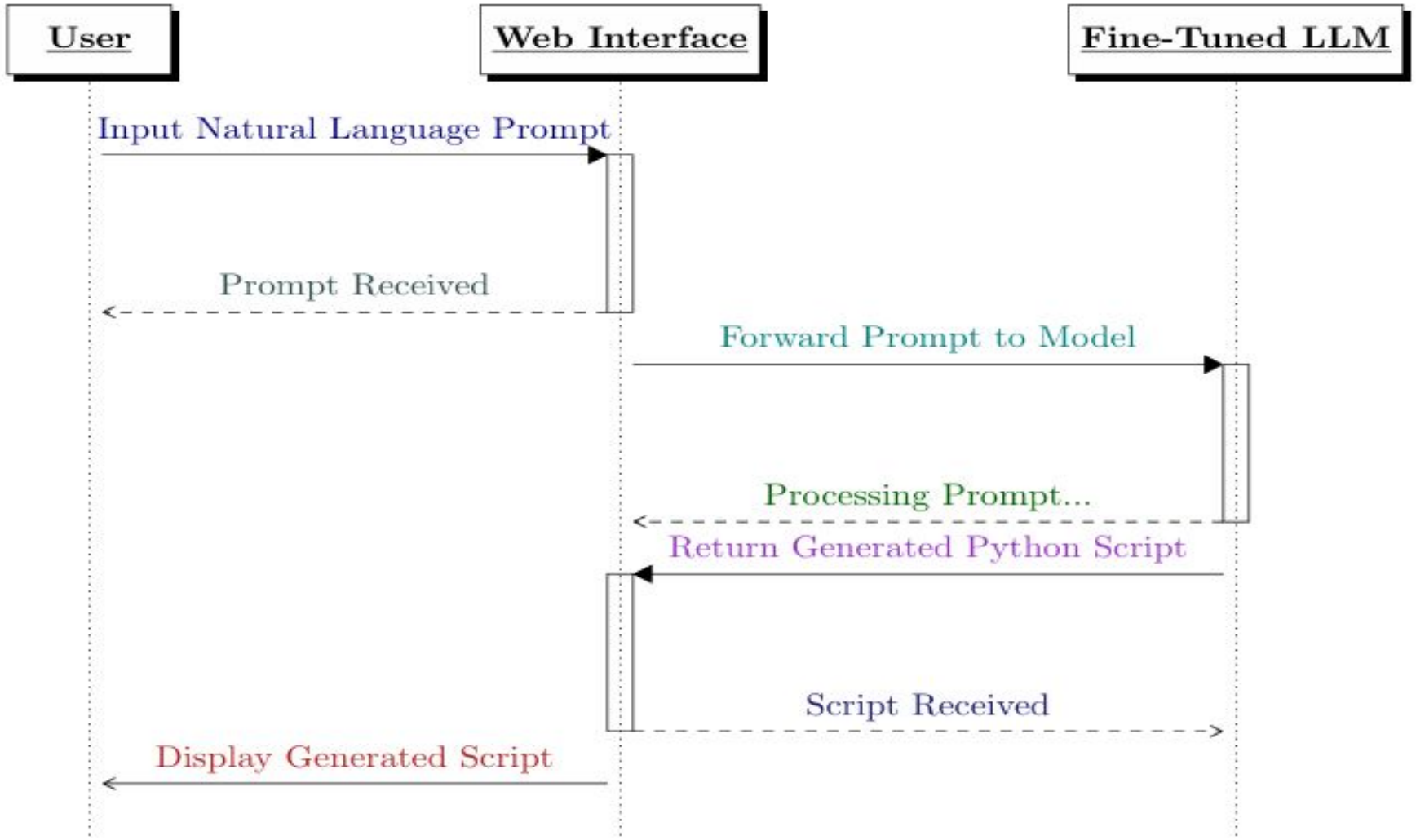
Use-Case Diagram:



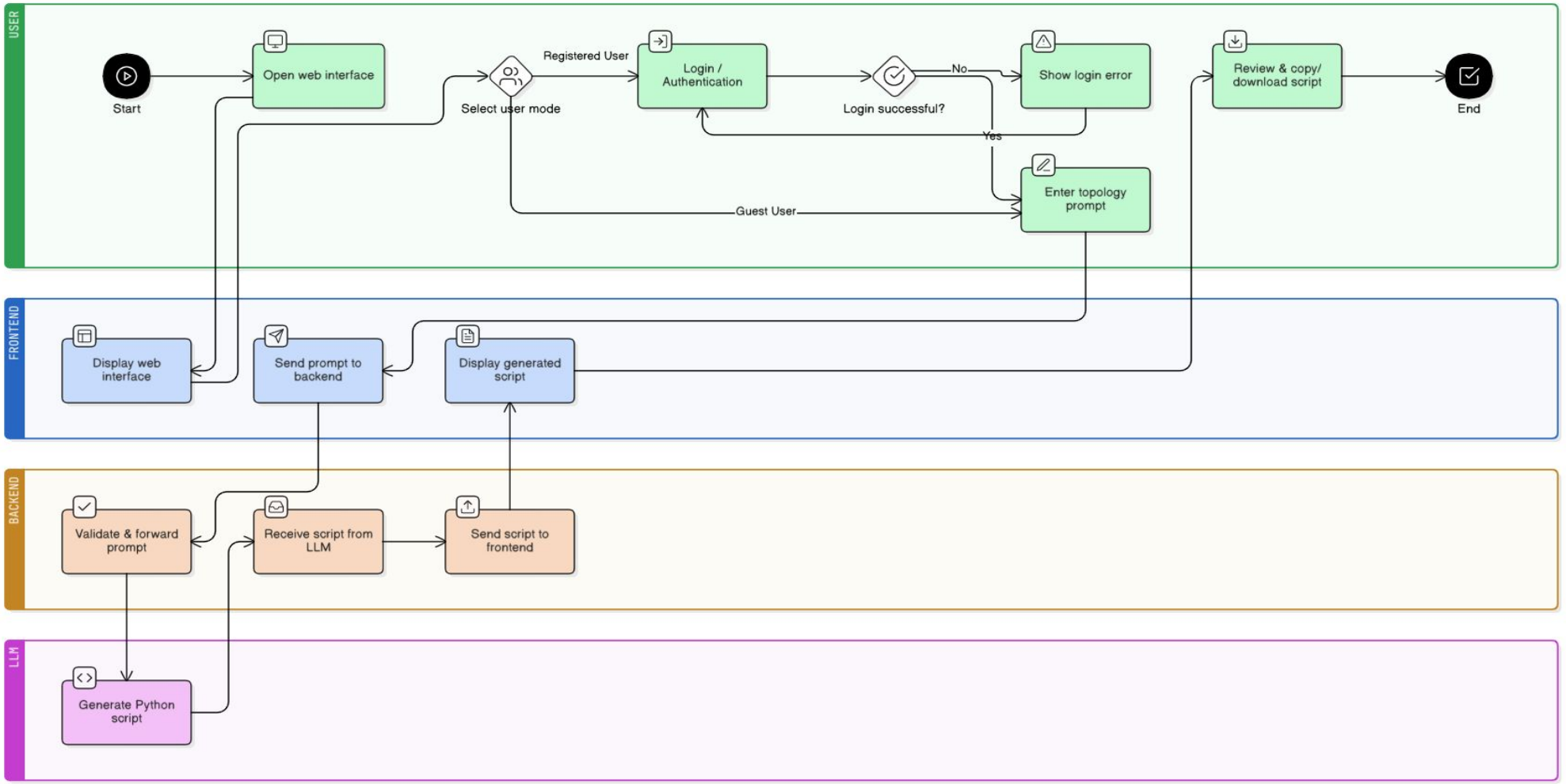
Activity Diagram



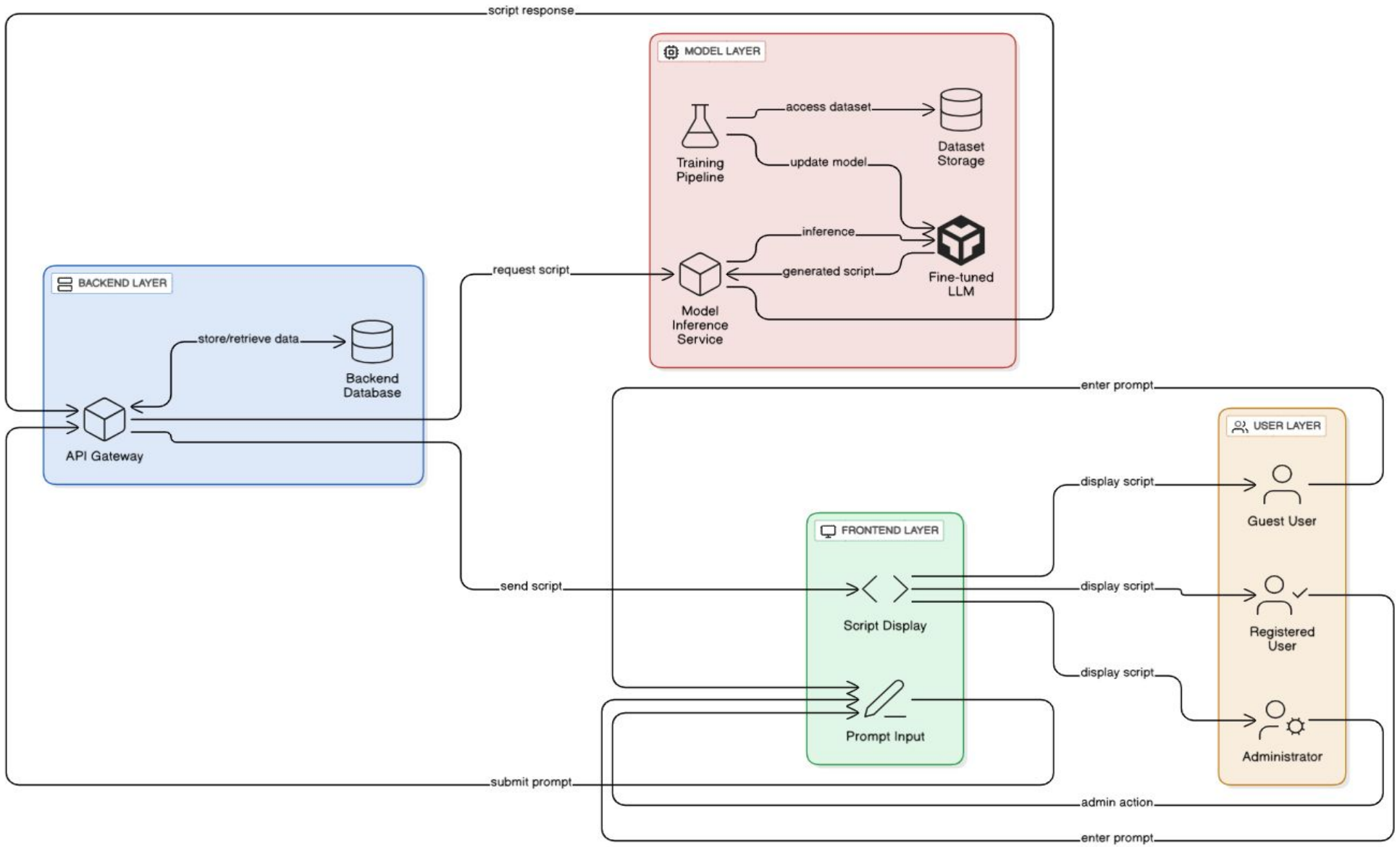
Sequence Diagram



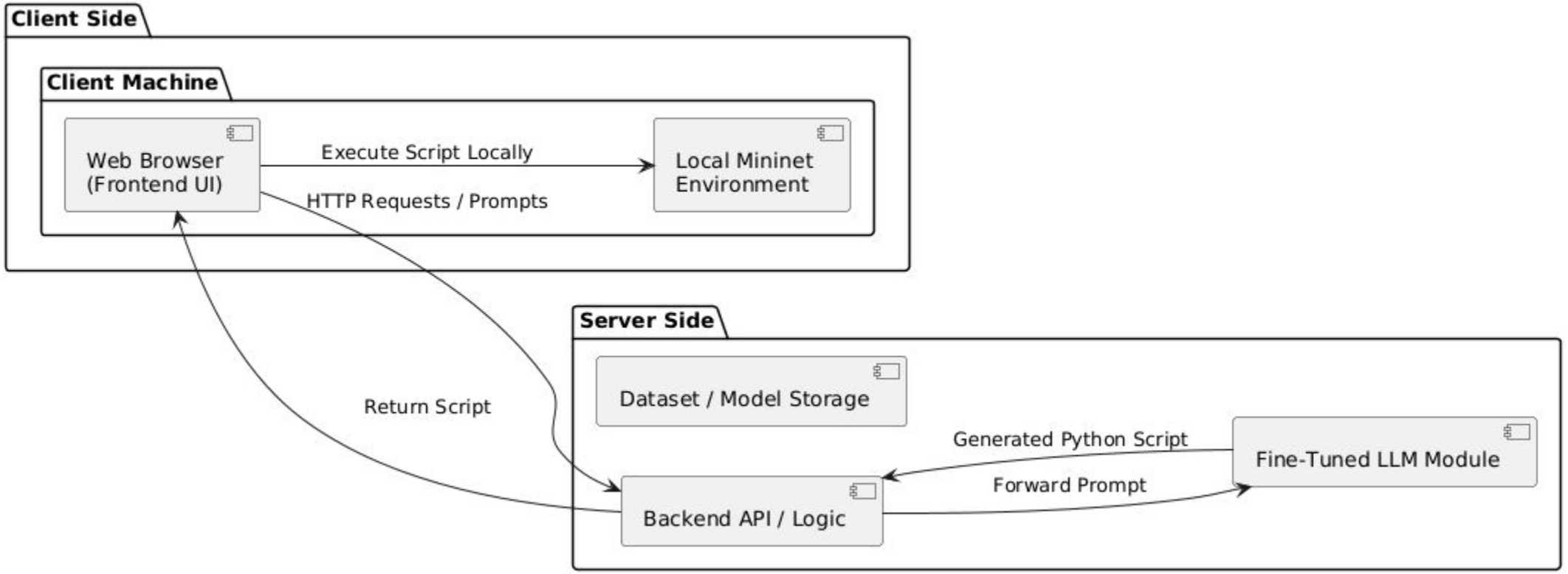
Flow Diagram



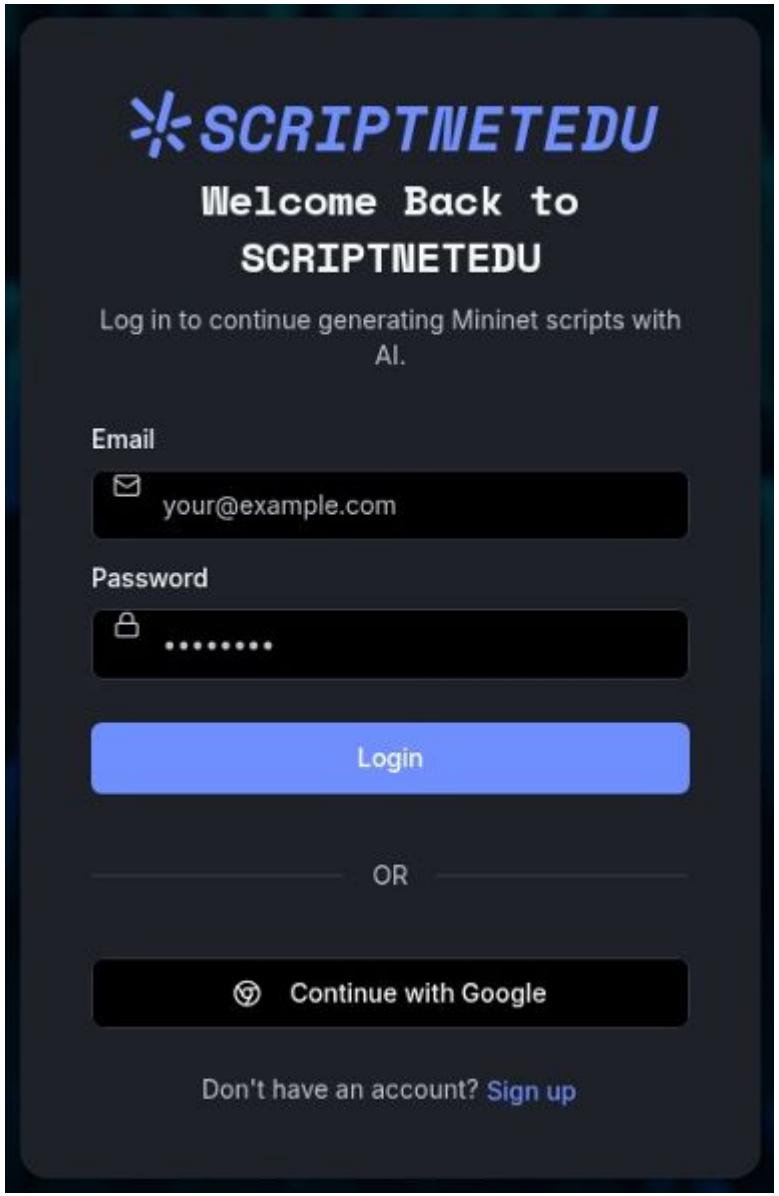
System Architecture Diagram

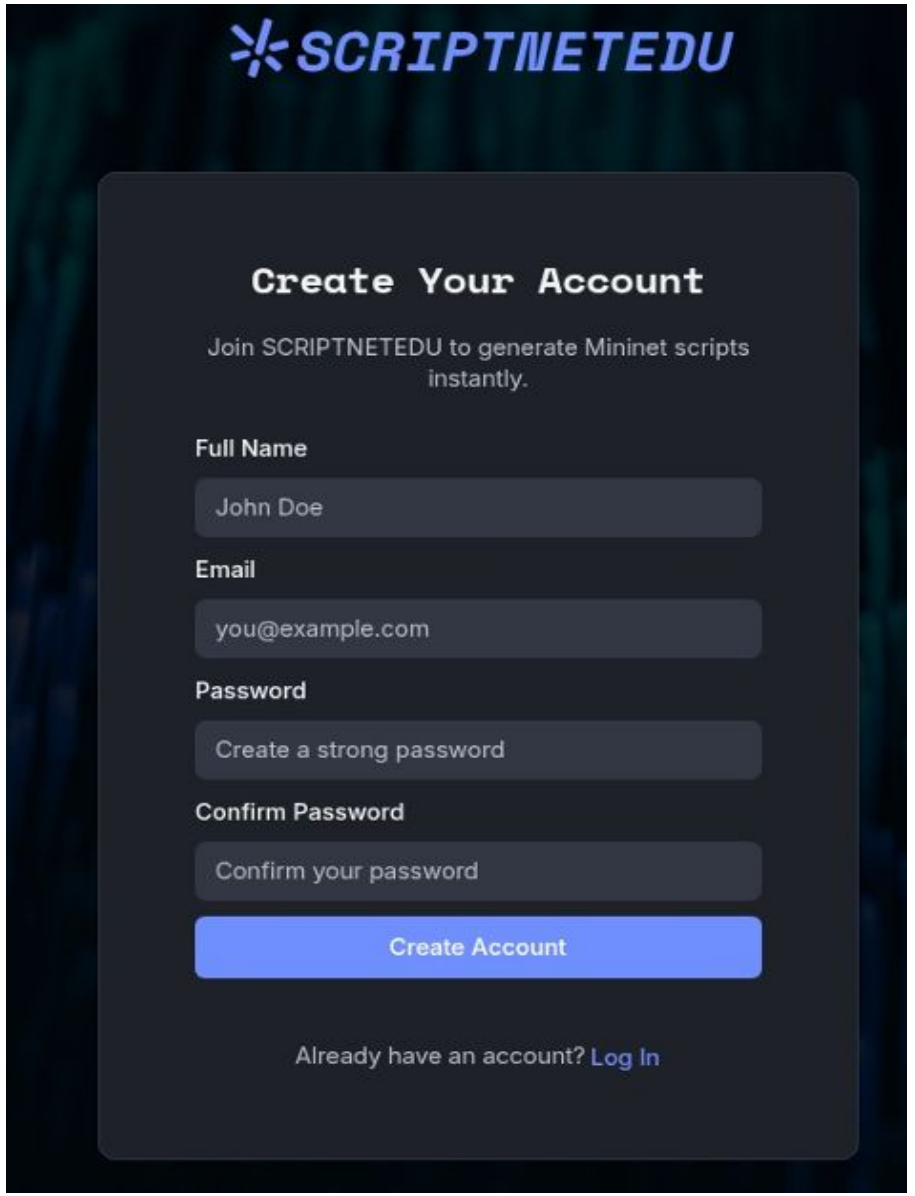


Deployment Diagram

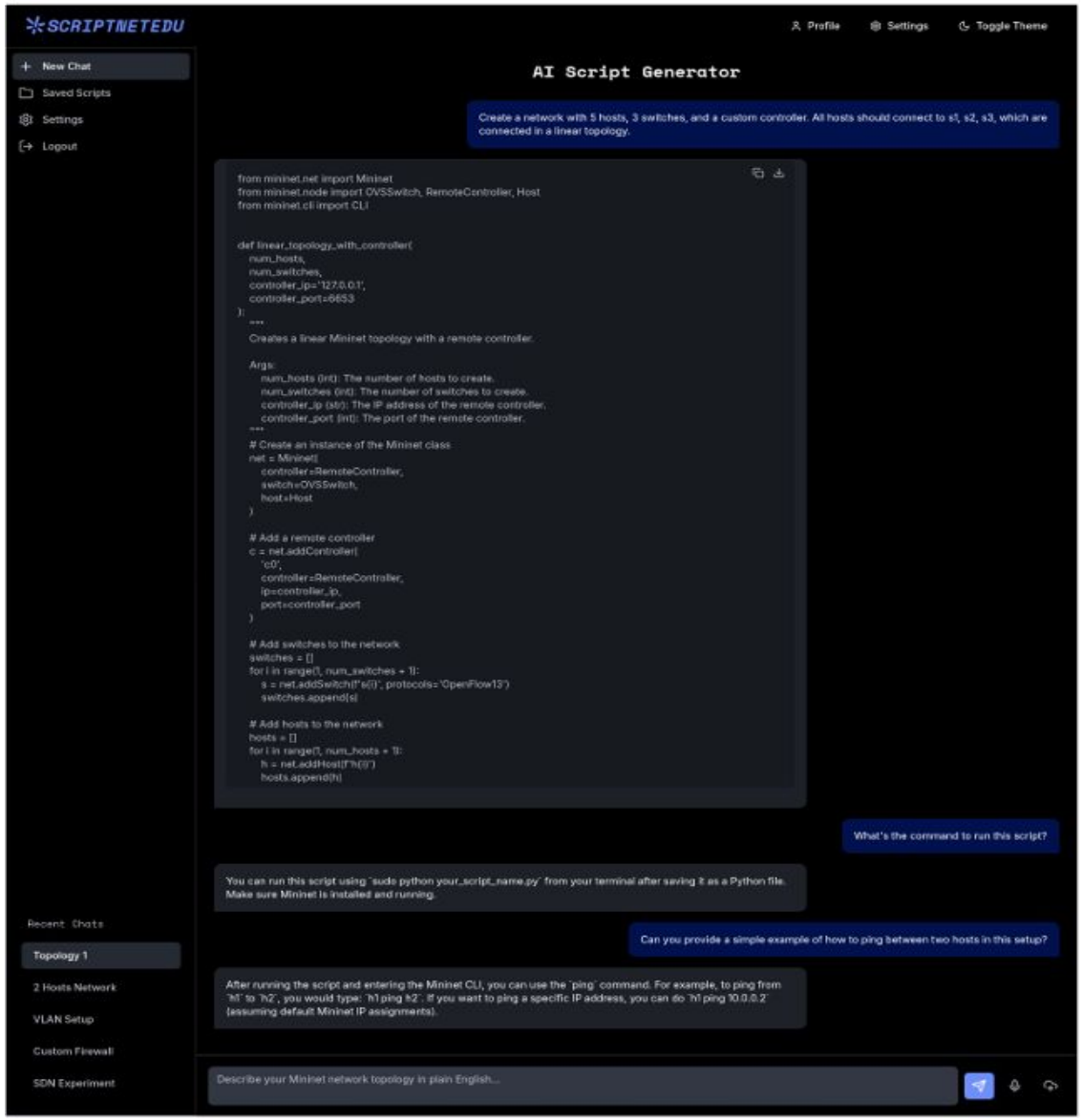


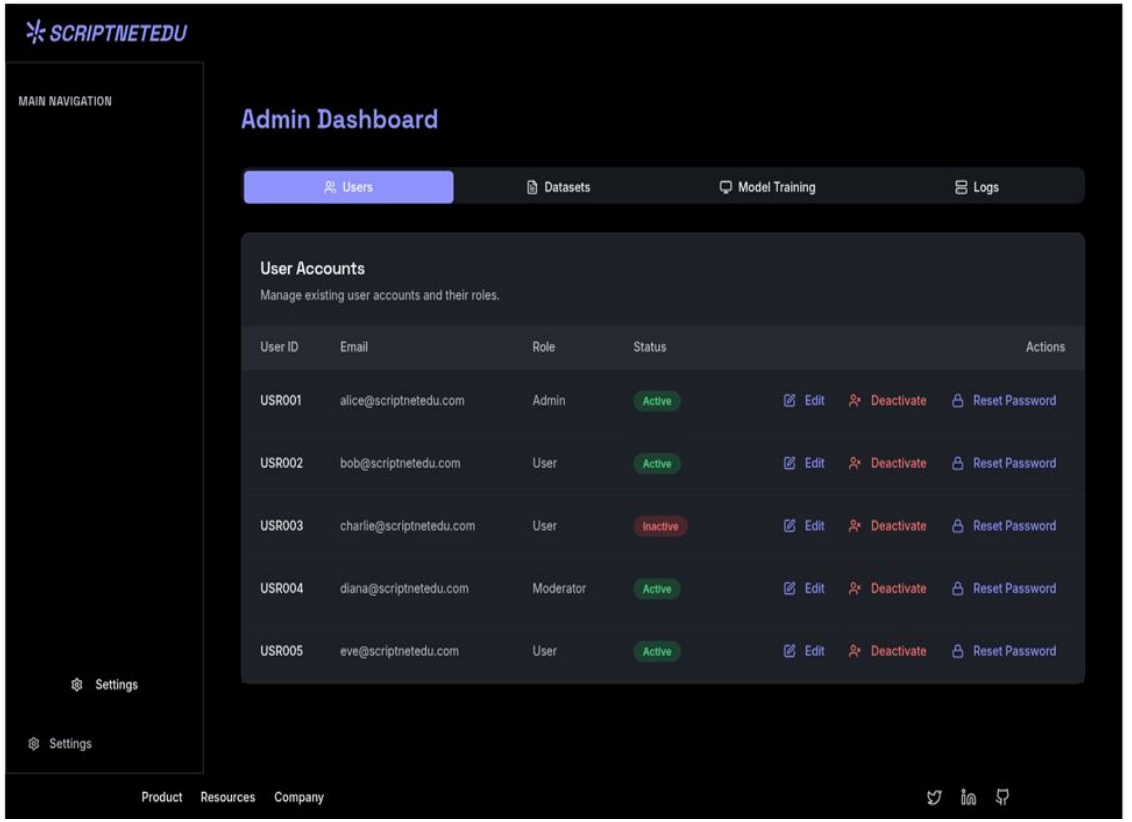
Web Design


SCRIPTNETEDU
Welcome Back to SCRIPTNETEDU
Log in to continue generating Mininet scripts with AI.
Email
your@example.com
Password
Login
OR
Continue with Google
Don't have an account? Sign up


SCRIPTNETEDU
Create Your Account
Join SCRIPTNETEDU to generate Mininet scripts instantly.
Full Name
John Doe
Email
you@example.com
Password
Create a strong password
Confirm Password
Confirm your password
Create Account
Already have an account? Log In

Web Design


SCRIPTNETEDU
Profile
Settings
Toggle Theme
New Chat
Saved Scripts
Settings
Logout
AI Script Generator
Create a network with 5 hosts, 3 switches, and a custom controller. All hosts should connect to s1, s2, s3, which are connected in a linear topology.
from mininet.net import Mininet
from mininet.node import OVSSwitch, RemoteController, Host
from mininet.cli import CLI
def linear_topology_with_controller(
num_hosts,
num_switches,
controller_ip='127.0.0.1',
controller_port=6653
):
"""
Creates a linear Mininet topology with a remote controller.
Args:
num_hosts (int): The number of hosts to create.
num_switches (int): The number of switches to create.
controller_ip (str): The IP address of the remote controller.
controller_port (int): The port of the remote controller.
"""
# Create an instance of the Mininet class
net = Mininet(
controller=RemoteController,
switch=OVSSwitch,
host=Host
)
# Add a remote controller
c = net.addController(
'c0',
controller=RemoteController,
ip=controller_ip,
port=controller_port
)
# Add switches to the network
switches = []
for i in range(1, num_switches + 1):
s = net.addSwitch('s{}'.format(i), protocols='OpenFlow13')
switches.append(s)
# Add hosts to the network
hosts = []
for i in range(1, num_hosts + 1):
h = net.addHost('h{}'.format(i))
hosts.append(h)
What's the command to run this script?
You can run this script using `sudo python your_script_name.py` from your terminal after saving it as a Python file. Make sure Mininet is installed and running.
Can you provide a simple example of how to ping between two hosts in this setup?
After running the script and entering the Mininet CLI, you can use the `pingall` command. For example, to ping from `h1` to `h2`, you would type: `h1 ping h2`. If you want to ping a specific IP address, you can do `h1 ping 10.0.0.2` (assuming default Mininet IP assignments).
Recent Chats
Topology 1
2 Hosts Network
VLAN Setup
Custom Firewall
SDN Experiment
Describe your Mininet network topology in plain English...


SCRIPTNETEDU
MAIN NAVIGATION
Admin Dashboard
Users
Datasets
Model Training
Logs
User Accounts
Manage existing user accounts and their roles.
User ID
Email
Role
Status
Actions
USR001
alice@scriptnetedu.com
Admin
Active
Edit
Deactivate
Reset Password
USR002
bob@scriptnetedu.com
User
Active
Edit
Deactivate
Reset Password
USR003
charlie@scriptnetedu.com
User
Inactive
Edit
Deactivate
Reset Password
USR004
diana@scriptnetedu.com
Moderator
Active
Edit
Deactivate
Reset Password
USR005
eve@scriptnetedu.com
User
Active
Edit
Deactivate
Reset Password
Settings
Settings
Product
Resources
Company

Test Cases

ID	Feature / Module	Test Description	Input / Action	Expected Output
TC-01	Model Input Validation	Check if LLM accepts natural language prompts correctly	"Create a network of 3 switches"	Prompt processed successfully
TC-02	Code Generation Accuracy	Verify if generated Python code is syntactically correct	Valid topology prompt	Executable Mininet script produced
TC-03	Topology Variation	Generate multiple structures with same specs	"3 hosts, 1 switch"	Distinct but valid topologies generated
TC-04	Dataset Load Test	Model trained dataset accuracy	Load diverse topology data	Model performance remains consistent
TC-05	Homepage Load	Website loads properly	Open site URL	Page loads swiftly
TC-06	Chatbot Interface	Ensure chatbot input/output fields appear correctly	Open Chatbot Page	Input and output visible
TC-07	User Login	Verify authentication	Enter valid credentials	Redirect to dashboard
TC-08	Responsive Design	Test UI on different screens	Resize browser / mobile view	Responsive layout adjusts correctly
TC-09	Connection Reliability	Ensure chatbot-LLM link stable	Trigger generation repeatedly	successful responses

Evaluation Metrics

Metric	Description	Measurement Method	Goal / Threshold
Code Accuracy (%)	Correctness of generated Mininet Python scripts	Manual + automated syntax tests	55-65%
Response Latency (s)	Time for LLM to respond to user prompt	Backend logs	approx.5 sec
Prompt Success Rate (%)	Valid outputs from total user prompts	Success / total prompts × 100	80-90%
Page Load Time (s)	Time taken for homepage to load	Browser Developer Tools	3 sec
Authentication Reliability (%)	Successful logins and signups under load	Load testing tools	pass
Cross-Browser Compatibility	Consistent UI rendering across browsers	Chrome, Firefox, Edge tests	100% pass
Error Recovery (%)	System's ability to handle errors without crash	Exception tracking logs	pass
Session Persistence	Retains login/session state after reload	Manual + cookie testing	100% persistence

The Challenge: LLM Inaccuracy in Network Generation

- LLMs struggle significantly with specialized, domain-specific code generation.
- Benchmarks for network simulation scripts (e.g., SIMCODE) show even state-of-the-art models achieve a very low baseline execution accuracy of only **~30%**.
- This highlights the inherent difficulty and high failure rate of generating complex, specialized code.
- This poor performance creates a significant "accuracy gap" compared to general Python tasks (where models can achieve ~64% accuracy).

C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, *"NetConfEval: Can LLMs Facilitate Network Configuration?," Proc. ACM Netw.*, vol. 2, no. CoNEXT2, Article 7, June 2024, doi: 10.1145/3656296.

T. Ahmed, M. M. Azwad, and S. Choudhury, *"SIMCODE: A Benchmark for Natural Language to ns-3 Network Simulation Code Generation,"* in *Proc. 50th IEEE Conf. Local Comput. Netw. (LCN)*, Sydney, Australia, Oct. 2025, forthcoming.

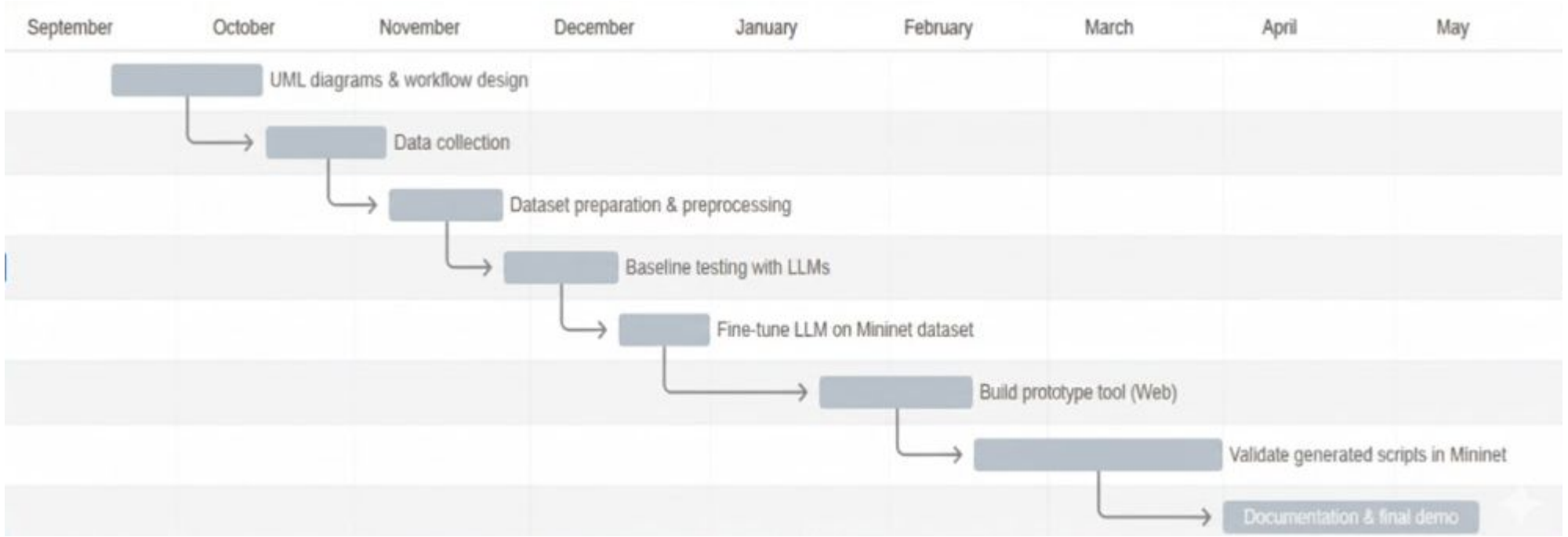
Dataset

Metric	Total
Number of Codes	150 (till now)

Source of Dataset:

- **Github**
- **University lab exercise (Cornell university etc..)**
- **Mininet official provided python scripts for network topologies**

Timeline


September
October
November
December
January
February
March
April
May
UML diagrams & workflow design
Data collection
Dataset preparation & preprocessing
Baseline testing with LLMs
Fine-tune LLM on Mininet dataset
Build prototype tool (Web)
Validate generated scripts in Mininet
Documentation & final demo

[1]. Großmann, M., & Schuberth, S. J. A. (2013). *Auto-Mininet: Assessing the Internet Topology Zoo in a Software-Defined Network Emulator*. Proceedings of the IEEE Conference on Network Simulation and Emulation.

[2]. Internet Topology Zoo Project. (2013). *Dataset and GraphML Resources for Network Topologies.*

[3]. Pal, C., Veena, S., Rustagi, R. P., & Murthy, K. N. B. (2014). *Implementation of Simplified Custom Topology Framework in Mininet.* International Journal of Computer Applications, 90(15), 1–6.

[4]. Kaur, K., Singh, J., & Ghumman, N. S. (2014). *Mininet as Software Defined Networking Testing Platform.* International Journal of Advanced Research in Computer and Communication Engineering, 3(5), 1–5.

[5]. Pakzad, F., Portmann, M., Tan, W. L., & Indulska, J. (2015). *Efficient Topology Discovery in OpenFlow-based Software Defined Networks.* IEEE Transactions on Network and Service Management, 12(1), 1–13.

[6]. Fontes, R., & Sampaio, P. N. M. (2015). *Visual Network Description: A Customizable GUI for the Creation of Software Defined Network Simulations.* Proceedings of the International Conference on Network Simulation Tools.

[7]. Bholebawa, I. Z., & Dalal, U. D. (2016). *Design and Performance Analysis of OpenFlow-Enabled Network Topologies Using Mininet.* International Journal of Computer Applications, 138(5), 25–30.

[8]. Nandi, S. K. (2016). *Topology Generators for Software Defined Network Testing.*

[9]. Laurito, A., Bonaventura, M., Astigarraga, M. E. P., & Castro, R. (2017). *TOPOGEN: A Network Topology Generation Architecture.* Proceedings of the IEEE Symposium on Network Management.

[10]. Noora, V. T., & Jinny, S. V. (2020). *Design of Different Topologies in SDN Controller.* International Research Journal of Engineering and Technology (IRJET), 7(9), 689–695.

[11]. Romanov, O. I., Saychenko, I. O., Marinov, A. I., & Skolets, S. S. (2021). *Research of SDN Network Performance Parameters Using Mininet Network Emulator.* Journal of Theoretical and Applied Information Technology, 99(8), 1907–1917.

[12]. Zacharis, A., Margariti, S. V., Stergiou, E., & Angelis, C. (2021). *Performance Evaluation of Topology Discovery Protocols in Software Defined Networks.* IEEE Access, 9, 71230–71242.

[13]. Kim, D.-W., Shin, G.-Y., et al. (2023). *Framework for Network Topology Generation and Traffic Prediction Analytics for Cyber Exercises.* Proceedings of the International Conference on Cybersecurity and Networks.